MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

VREMERE ADRIAN, FAF-232

Report

Laboratory Work n.4.1

Dual Actuator Control — Binary Relay & Analog PWM

on Embedded Systems

Checked by:
Martîniuc Alexei, *univ. asist.*
FCIM, UTM

Chişinău – 2026

1. Domain Analysis

1.1. Objective of the Laboratory Work

The objective of this laboratory work is to design and implement a modular dual-actuator control system on an Arduino Mega 2560 microcontroller. The system controls two types of actuators — a binary actuator (relay) and an analog actuator (PWM-driven motor/LED) — through serial commands, with real-time signal conditioning and status reporting on an LCD display.

This lab addresses Varianta C of the assignment (100% score), which requires simultaneous control of both a binary and an analog actuator, each with dedicated signal conditioning pipelines and unified display reporting. The key learning objectives include understanding the principles of actuator control in embedded systems, implementing counter-based debouncing for binary signals, applying ramping and filtering techniques for smooth analog control, and coordinating multiple real-time tasks using FreeRTOS.

1.2. Problem Definition

(Original task statement in Romanian — Section 5.2 from the lab manual)

> Pentru un actuator binar bec iluminare prin releu, sau altul convenit cu mentorul să se realizeze o aplicaţie pentru MCU care va interpreata comenzi de control de la utilizator prin STDIO (serial sau keypad), va controla actuatorul afişând starea curentă pe un display LCD.
>
> Varianta C: Implementarea sistem de control a doi actuatori, unul binar şi unul analogic, cu afişarea valorilor şi alertelor de la ambii actuatori pe display LCD.

The task requires the development of a microcontroller application with the following capabilities:

1. **Binary actuator control:** receive ON/OFF commands from the serial terminal, apply counter-based debouncing to prevent false switching, and drive a relay module that controls an external load (simulated lamp).

2. **Analog actuator control:** receive speed commands (0–100%) from the serial terminal, apply ramping for smooth transitions and signal conditioning (saturation, median filtering, EWMA) to eliminate noise, and drive a PWM output controlling a motor or LED.

3. **Display and reporting:** periodically display the state of both actuators on a 16x2 LCD and print structured reports to the serial terminal, including debounce counters and conditioning diagnostics.

4. **Multi-task architecture:** use FreeRTOS to run command parsing, binary control, analog control, and display as independent tasks with protected shared state via mutex.

1.3. Technical Requirements

Functional Requirements

FR1 The system shall accept `ON` and `OFF` commands to control the binary actuator (relay).

FR2 The system shall accept `SPEED N` commands (N = 0–100) to set the analog actuator duty cycle.

FR3 The system shall accept a `STATUS` command to query the current state of both actuators.

FR4 The binary actuator command shall pass through a counter-based debounce filter (5 confirmations at 100 ms intervals) before the relay state changes.

FR5 The analog actuator setpoint shall be ramped at a maximum rate of 5%/cycle and processed through a signal conditioning pipeline (saturation 0–100%, median filter, EWMA).

FR6 The LCD shall display the relay state, PWM duty cycle, debounce counter, and raw setpoint, updated every 500 ms.

FR7 A serial report with both actuator states and conditioning diagnostics shall be printed every 500 ms.

Non-Functional Requirements

NFR1 The system shall respond to serial commands with a latency below 100 ms.

NFR2 All shared variables between FreeRTOS tasks shall be protected by a mutex to prevent data races.

NFR3 The software shall be modular, with separate library modules for each hardware driver and signal conditioner.

NFR4 Pin assignments shall be defined in a single configuration header for consistency across code, simulation, and documentation.

1.4. Test Scenarios and Validation Criteria

1. **TC1 — Binary ON command:** send “ON” via serial. Expected: after 5 debounce cycles (500 ms), relay activates, green LED turns on, LCD shows “Rly:ON”.
2. **TC2 — Binary OFF command:** send “OFF” after relay is ON. Expected: after 5 debounce cycles, relay deactivates, green LED turns off.
3. **TC3 — Rapid toggle rejection:** send “ON” then immediately “OFF” before debounce completes. Expected: relay state does not change (debounce counter resets).

4. **TC4 — Analog speed command:** send “SPEED 75”. Expected: PWM ramps gradually to 75%, signal conditioning smooths the transition, yellow LED turns on.

5. **TC5 — Analog ramp verification:** send “SPEED 100” from 0%. Expected: duty increases by at most 5%/cycle (100 ms), reaching 100% in approximately 2 seconds.

6. **TC6 — Analog saturation:** send “SPEED 150”. Expected: duty clamped to 100%.

7. **TC7 — Status command:** send “STATUS”. Expected: serial prints current relay state and PWM duty cycle.

8. **TC8 — LCD display:** observe LCD during operation. Expected: line 1 shows relay state and PWM percentage, line 2 shows debounce counter and raw setpoint.

9. **TC9 — Unknown command:** send “BLINK”. Expected: serial prints error message with usage help.

10. **TC10 — Dual actuator simultaneous:** send “ON” and “SPEED 50” in sequence. Expected: both actuators operate independently and are displayed on LCD.

1.5. Used Technologies

Actuator Control in Embedded Systems

Actuators are devices that convert electrical energy into physical action. In embedded systems, actuators are the output stage that bridges the digital world of the microcontroller with the physical environment. They are broadly classified into two categories based on their control signal: binary actuators and analog actuators.

Binary actuators operate in only two states — ON and OFF. The most common example is a relay, which is an electrically operated switch that uses an electromagnetic coil to mechanically open or close a circuit. Relays are widely used to control high-power loads (lamps, heaters, motors) from low-power microcontroller outputs. The relay coil typically requires more current than a GPIO pin can provide, so a transistor driver or dedicated relay module with an optocoupler is used for isolation and signal amplification. A critical consideration when controlling binary actuators is signal debouncing: commands arriving from noisy inputs (buttons, serial interface with potential repeated transmissions) must be validated through persistent confirmation before the actuator state is changed, preventing false switching that could damage equipment or create hazardous conditions.

Analog actuators accept a continuously variable control signal, allowing proportional control of physical quantities such as speed, position, brightness, or temperature. In microcontroller systems, the most common method for driving analog actuators is Pulse Width Modulation (PWM). PWM generates a square wave at a fixed frequency, and the duty cycle (the fraction of the period during which the signal is HIGH) determines the average power delivered to the actuator. For a DC

motor, varying the duty cycle controls the speed; for an LED, it controls brightness. The relationship between duty cycle and average voltage is given by $V_{avg} = D \times V_{supply}$, where D is the duty cycle (0.0 to 1.0). Signal conditioning for analog actuators includes saturation (clamping values to safe limits), filtering (removing noise from the command signal), and ramping (limiting the rate of change to prevent mechanical shock or current surges during rapid transitions).

Counter-Based Debouncing

Counter-based debouncing is a software technique for filtering noisy binary signals. Instead of accepting a state change immediately, the algorithm maintains a counter that increments when the input is HIGH and decrements when LOW. The output only transitions to ON when the counter reaches a maximum threshold, and to OFF when it reaches zero. Intermediate counter values retain the previous stable state. This approach is more robust than simple time-based debouncing because it can reject short bursts of noise while still responding quickly to sustained state changes. The number of required confirmations and the sampling interval together determine the minimum pulse duration that the filter will pass.

Signal Conditioning for Analog Actuators

The signal conditioning pipeline for analog actuator commands applies three sequential stages to the raw command value. First, saturation clamps the value to the physically valid range (0–100% for a duty cycle), rejecting out-of-range commands. Second, a median filter removes impulsive noise by selecting the median from a sliding window of recent samples, effectively eliminating outlier values without distorting the signal shape. Third, an Exponentially Weighted Moving Average (EWMA) smooths the median-filtered value, reducing residual fluctuations while preserving responsiveness. Before entering this pipeline, a ramping stage limits the rate of change per control cycle, ensuring that the actuator transitions smoothly between setpoints.

FreeRTOS Real-Time Operating System

FreeRTOS is a real-time operating system kernel designed for microcontrollers and small embedded systems. It provides preemptive multitasking with priority-based scheduling, allowing multiple independent tasks to run concurrently on a single processor. Each task has its own stack and execution context, and the scheduler switches between tasks based on priority and timing constraints. FreeRTOS provides synchronization primitives such as mutexes and semaphores for protecting shared resources between tasks. In this lab, a mutex protects the shared command targets and actuator state variables that are written by the command task and read by the actuator and display tasks.

PlatformIO Build System

PlatformIO is a professional open-source ecosystem for embedded development. It provides project management, library dependency resolution, multi-board support, and integrated

build/upload/monitor tools. In this lab, PlatformIO is used within VS Code to compile, upload, and monitor the application on the Arduino Mega 2560.

Wokwi Simulator

Wokwi is an online and VS Code-integrated simulator for embedded systems that supports Arduino, ESP32, and other popular platforms. It allows testing of the application in a virtual environment before deploying to physical hardware, providing a simulated serial terminal, virtual LEDs, relay modules, and I2C LCD displays.

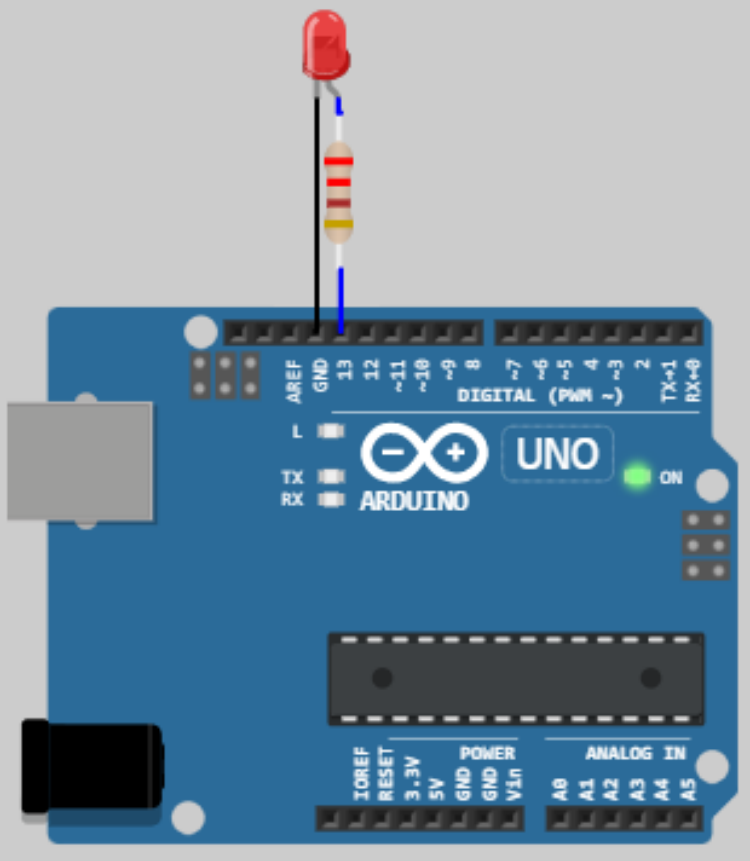

Figure 1.1 *– Wokwi simulator interface for embedded circuit simulation*

1.6. Hardware Components

Arduino Mega 2560

The Arduino Mega 2560 is a microcontroller development board based on the ATmega2560 AVR chip. It features 54 digital I/O pins (15 of which support PWM output), 16 analog inputs, 4 hardware UART ports, a 16 MHz crystal oscillator, and USB connectivity. The board operates at 5 V with 8 KB SRAM and 256 KB flash memory. In this lab, the Mega 2560 serves as the central controller, running FreeRTOS tasks that manage command parsing, actuator control, and display reporting.

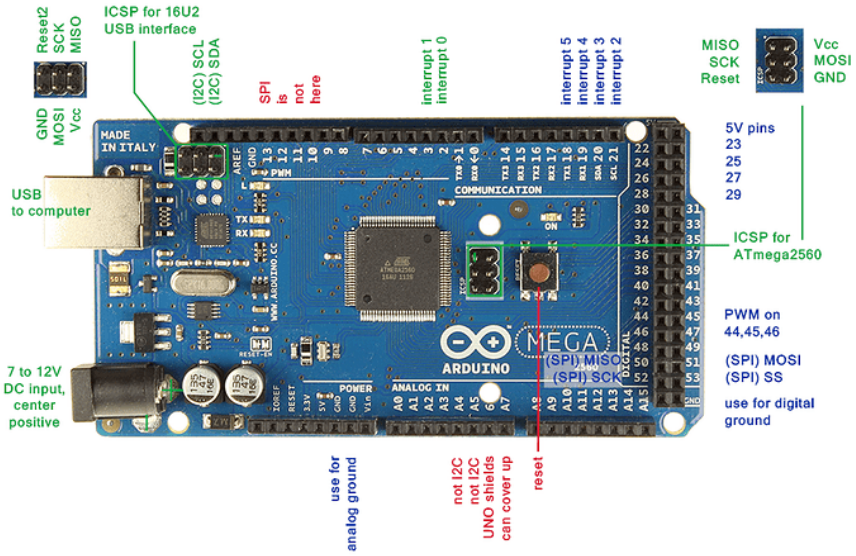

Figure 1.2 *– Arduino Mega 2560 development board*

Relay Module

The relay module is a single-channel electromechanical switch controlled by a digital GPIO pin. It uses an optocoupler for electrical isolation between the microcontroller and the relay coil, and includes a flyback diode for coil spike protection. The module has three output terminals: COM (common), NO (normally open), and NC (normally closed). When the control pin is driven HIGH, the relay coil energizes and the COM-NO circuit closes, allowing current to flow through the external load. When driven LOW, the relay deactivates and COM-NO opens.

In this lab, the relay module is connected to pin 7 of the Arduino Mega 2560 and controls a simulated lamp (LED in Wokwi). The relay operates at 5 V with a coil current of approximately 70 mA, drawn from the board's 5 V supply through the module's built-in driver circuit.

[Image pending — save to resources/DomainAnalysis/Hardware-Components/relay_module.png]

Figure 1.3 *– Single-channel relay module with optocoupler isolation*

LEDs and Resistors

Three LEDs are used in this lab for status indication and load simulation. A green LED on pin 12 mirrors the relay state (ON when relay is active), providing visual feedback for the binary actuator. A yellow LED on pin 11 indicates analog actuator activity (ON when duty cycle exceeds 1%). A blue LED on pin 9 (PWM-capable) simulates the analog actuator load — in a physical system, this would be replaced by a DC motor with an appropriate driver circuit. Each LED is connected through a 220 Ω current-limiting resistor to keep the forward current within safe limits ($I \approx 13.6$ mA at 5 V).

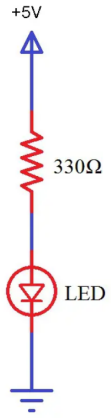

Figure 1.4 *– Standard 5mm LED used as status indicator*

LCD 1602 I2C Display

The LCD 1602 is a 16-column, 2-row character display that communicates with the microcontroller over the I2C bus (address 0x27). An I2C backpack module reduces the required connections from 16 parallel pins to just 4 wires (VCC, GND, SDA, SCL). The display is connected to the Arduino Mega's I2C pins (SDA on pin 20, SCL on pin 21) and is used to show real-time status of both actuators, including relay state, PWM duty cycle, debounce counter, and raw setpoint.

Figure 1.5 *– LCD 1602 display with I2C backpack module*

1.7. Software Components

Visual Studio Code with PlatformIO

Visual Studio Code with the PlatformIO IDE extension provides the complete development environment for this project. PlatformIO handles project configuration via `platformio.ini`, automatic toolchain management for the AVR target, library dependency resolution (FreeRTOS, LiquidCrystal_I2C), and integrated serial monitoring for testing commands and viewing reports.

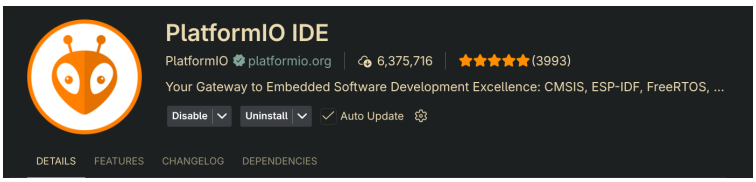

Figure 1.6 *– PlatformIO IDE extension in Visual Studio Code*

1.8. System Architecture and Justification

The system employs a layered software architecture combined with a FreeRTOS multi-task design. The architecture separates hardware drivers (relay, PWM, LED, LCD) from signal conditioning logic (debouncing, ramping, filtering) and application-level task orchestration.

The choice of FreeRTOS over the bare-metal TaskScheduler used in earlier labs is justified by the need for mutex-protected shared state between the command parser and actuator tasks. FreeRTOS provides preemptive scheduling with configurable task priorities, ensuring that the command parsing task (highest priority) can interrupt lower-priority display updates to maintain responsive command handling within the 100 ms latency requirement.

The signal conditioning approach is tailored to each actuator type. For the binary actuator, counter-based debouncing (5 confirmations at 100 ms intervals = 500 ms minimum transition time) prevents false switching while remaining responsive to sustained commands. For the analog actuator, the combination of ramping (5%/cycle rate limit) and the three-stage conditioning pipeline (saturation, median, EWMA) provides smooth, stable control that protects mechanical actuators from sudden load changes.

1.9. Case Study: Industrial Relay Control Systems

Relay-based control systems are ubiquitous in industrial automation. HVAC (Heating, Ventilation, and Air Conditioning) systems use relays to switch compressors, fans, and heating elements based on thermostat signals. Manufacturing production lines employ relay logic for sequencing conveyor belts, pneumatic actuators, and safety interlocks. Building management systems use relays to control lighting circuits, access gates, and fire suppression systems.

In all these applications, signal conditioning and debouncing are critical safety requirements. A false relay activation in an industrial oven or a medical sterilizer could cause overheating, while a false deactivation in a safety interlock could expose workers to hazardous conditions. The counter-based debouncing technique implemented in this lab mirrors the approach used in industrial programmable logic controllers (PLCs), where input filters with configurable persistence thresholds are standard practice for ensuring reliable actuator control.

2. Design

2.1. System Components

The dual actuator control system consists of the following components, each with a clearly defined role in the architecture:

1. **Arduino Mega 2560** — central microcontroller running FreeRTOS, hosting all tasks and coordinating communication between modules.
2. **Relay Module** — binary actuator driven by digital pin 7, switches an external load (lamp) ON/OFF.
3. **PWM Output (Pin 9)** — analog actuator interface generating a variable duty cycle signal for motor/LED control.
4. **Green LED (Pin 12)** — status indicator mirroring the relay state.
5. **Yellow LED (Pin 11)** — status indicator for analog actuator activity.
6. **Blue LED (Pin 9)** — load simulation for the analog actuator (PWM-driven brightness).
7. **LCD 1602 I2C** — 16x2 character display showing both actuator states, debounce counters, and setpoints.
8. **Serial Terminal (UART0)** — command input interface and diagnostic report output.
9. **BinaryConditioner** — software module implementing counter-based debouncing for the relay command.
10. **SignalConditioner** — software module providing saturation, median filtering, and EWMA for the analog command.
11. **FreeRTOS Scheduler** — manages four concurrent tasks with priority-based preemptive scheduling.

2.2. Structural Diagram

The structural diagram in *Figure 2.1* shows how the hardware and software components interact. The serial terminal feeds commands to the Command Task, which updates shared state variables. The Binary Task and Analog Task read these variables (protected by a mutex), apply their respective conditioning pipelines, and drive the hardware actuators. The Display Task reads all state and updates the LCD and serial output.

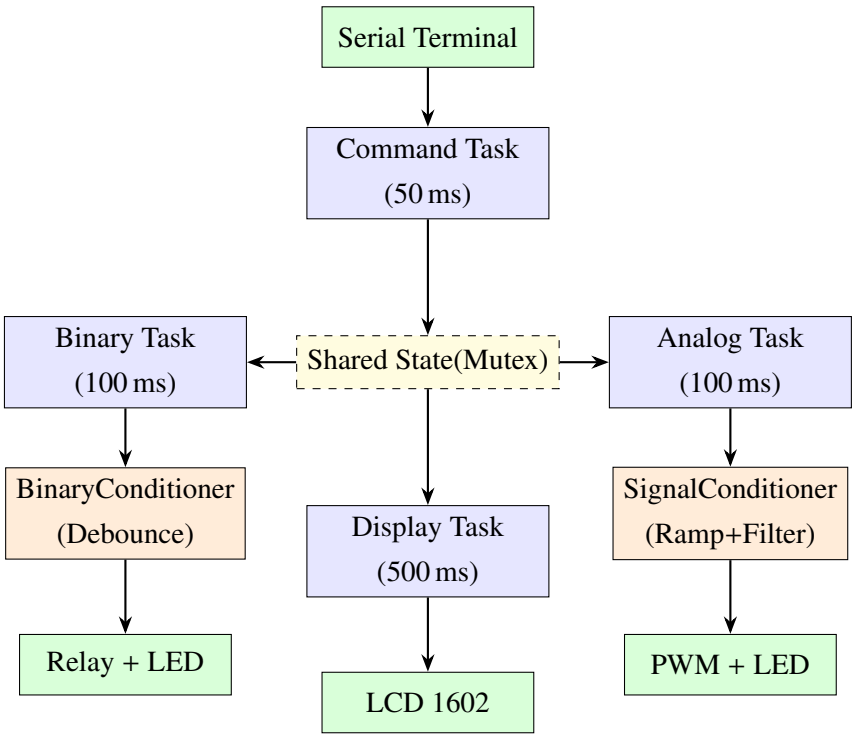

***Figure 2.1** – System structural diagram showing task interactions and data flow*

2.3. Layered Software Architecture

The software follows a layered architecture that separates concerns across five abstraction levels, as shown in *Figure 2.2*. The Application Layer (APP) contains the FreeRTOS tasks that orchestrate the system behavior. The Service Layer (SRV) provides signal conditioning algorithms (BinaryConditioner, SignalConditioner) and command parsing logic. The ECU Abstraction Layer (ECAL) handles STDIO-to-UART redirection and I2C LCD communication. The Microcontroller Abstraction Layer (MCAL) wraps low-level GPIO, PWM, and I2C peripheral access through the Relay, PwmActuator, Led, and LcdDisplay drivers. The Hardware Layer (HW) represents the physical ATmega2560 and its connected peripherals.

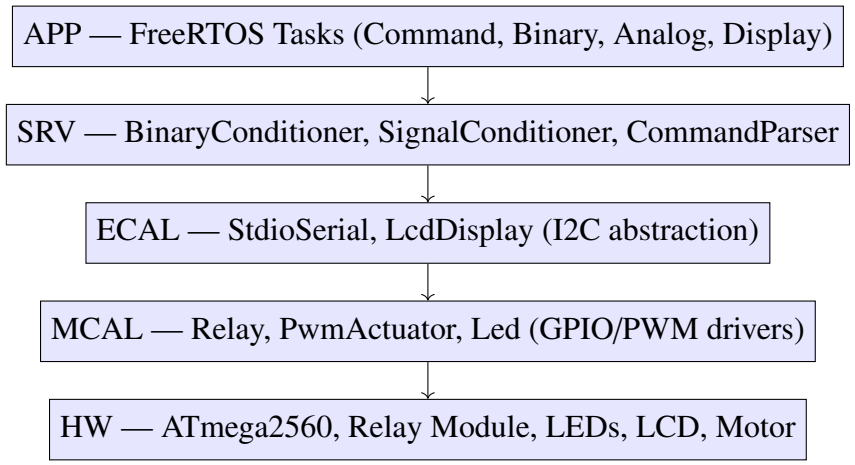

***Figure 2.2** – Layered software architecture for the dual actuator control system*

2.4. Hardware-Software Interface Architecture

For each hardware interface, the layered architecture provides a clear mapping from physical connection to software API:

- **Relay interface:** GPIO pin 7 → MCAL `Relay` class (`turnOn()`, `turnOff()`, `isOn()`) → SRV `BinaryConditioner` → APP Binary Task.
- **PWM interface:** PWM pin 9 → MCAL `PwmActuator` class (`setDutyCycle()`, `getDutyCycle()`) → SRV `SignalConditioner` with ramping → APP Analog Task.
- **LED interfaces:** GPIO pins 11, 12 → MCAL `Led` class (`turnOn()`, `turnOff()`) → APP actuator tasks.
- **LCD interface:** I2C bus (SDA=20, SCL=21) → ECAL `LcdDisplay` class (`showTwoLines()`, `printLine()`) → APP Display Task.
- **Serial interface:** UART0 (pins 0/1) → ECAL `StdioSerial` (`printf()`, `Serial.read()`) → APP Command Task.

2.5. Behavioral Diagrams

Application Main Flowchart

The main application flow, shown in *Figure 2.3*, illustrates the initialization sequence and the FreeRTOS task creation. After setup completes, the FreeRTOS scheduler takes over and the four tasks run concurrently according to their priorities and periods.

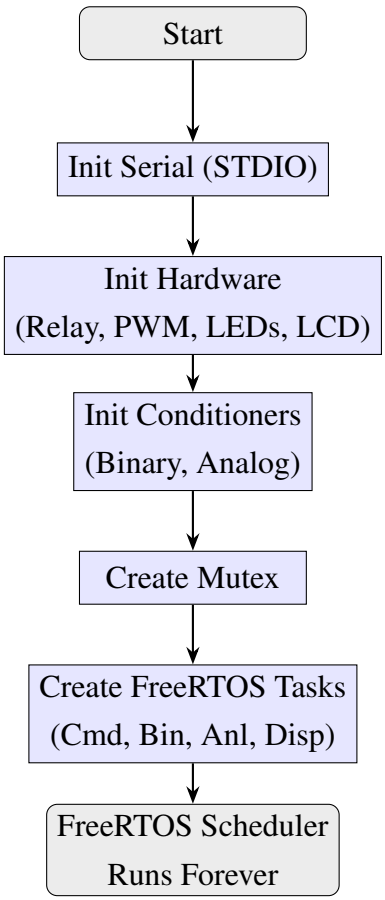

Figure 2.3 *– Application initialization flowchart*

Command Task Flowchart

The command task, shown in *Figure 2.4*, runs every 50 ms and reads all available characters from the serial buffer. When a complete line is received (terminated by newline), it parses the command and updates the shared state under mutex protection.

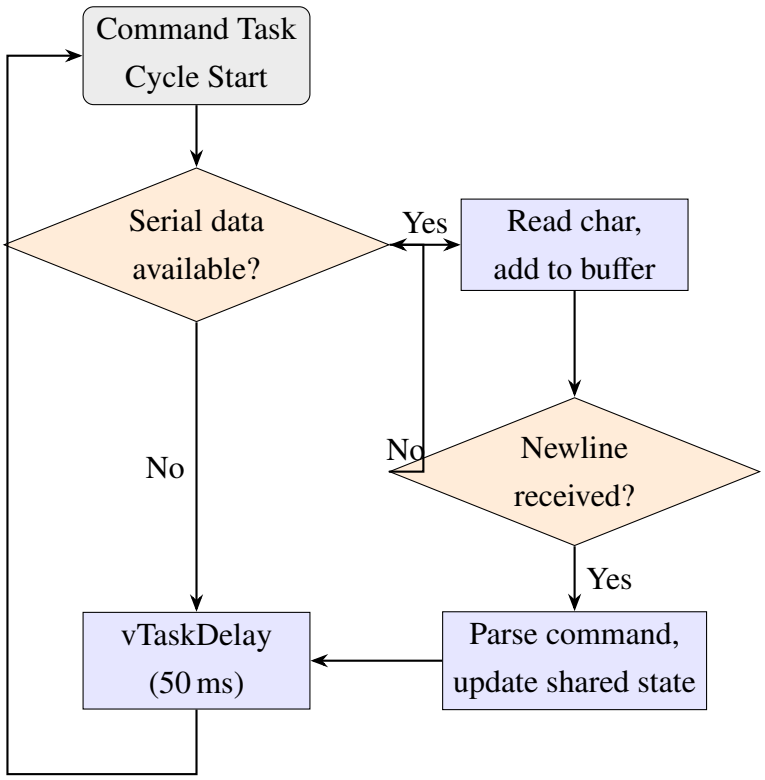

Figure 2.4 *– Command task flowchart*

Binary Actuator Task Flowchart

The binary actuator task, shown in *Figure 2.5*, reads the command target, processes it through the debounce filter, and drives the relay and status LED based on the stable output.

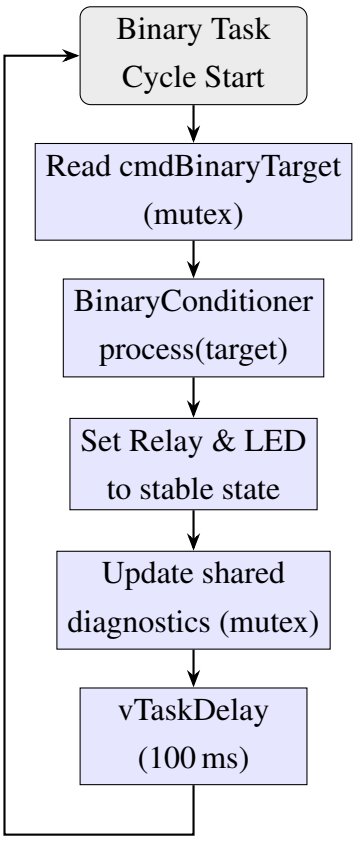

Figure 2.5 *– Binary actuator task flowchart*

Analog Actuator Task Flowchart

The analog task, shown in *Figure 2.6*, reads the target setpoint, applies ramping to limit the rate of change, processes the ramped value through the signal conditioning pipeline, and writes the conditioned output to the PWM actuator.

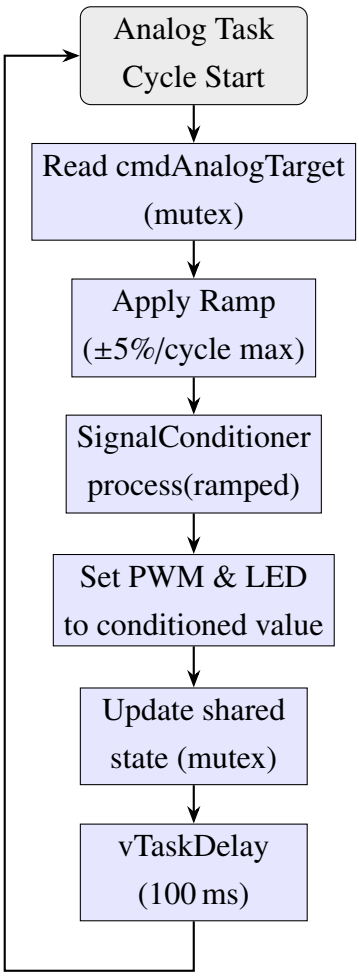

Figure 2.6 *– Analog actuator task flowchart*

Binary Conditioner State Diagram

The debounce filter operates as a simple state machine with two stable states (OFF and ON) and transitional behavior governed by the counter, as shown in *Figure 2.7*.

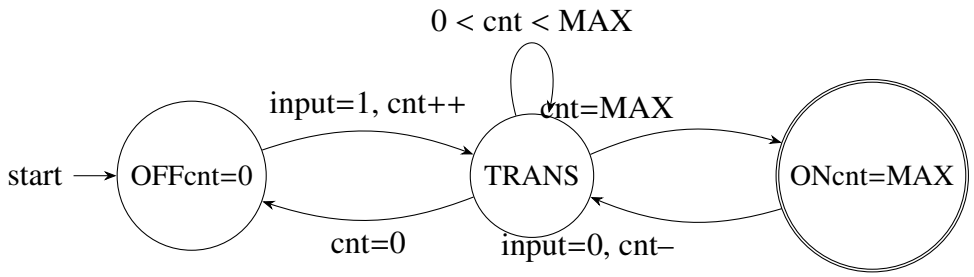

Figure 2.7 *– Binary conditioner (debounce) state diagram*

2.6. Electrical Schematic

The circuit schematic in *Figure 2.8* shows the complete wiring of the dual actuator system. The relay module is connected to pin 7 with its load (white LED) on the normally-open contact. The PWM actuator LED is on pin 9. Status LEDs on pins 11 and 12 provide visual feedback. The LCD communicates via I2C on pins 20/21.

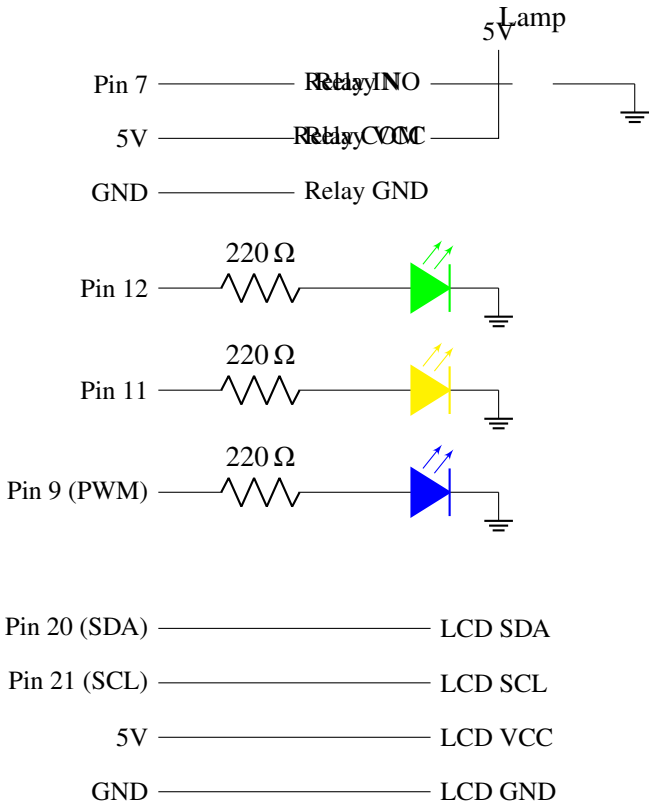

***Figure 2.8** – Complete circuit schematic of the dual actuator control system*

2.7. Wokwi Simulation Circuit

[Screenshot pending — save to resources/Design/ElectricalSchematics/wokwi_circuit.png]

***Figure 2.9** – Wokwi simulation circuit for the dual actuator control system*

2.8. Project Structure

The project files are organized to reflect the layered architecture, with each module in its own library directory:

***Listing 2.1** – Project directory structure for Lab 4.1*

```
labs/
  src/main.cpp                    # Lab selector (ifdef LAB4_1)
  lab/lab4_1/
    lab4_1_main.h                 # Entry point header
    lab4_1_main.cpp               # FreeRTOS tasks & orchestration
    lab4_1_config.h               # Pin mapping & timing constants
  lib/
    Relay/Relay.h, .cpp           # MCAL: relay driver
    PwmActuator/PwmActuator.h,.cpp # MCAL: PWM output driver
    Led/Led.h, .cpp               # MCAL: LED driver (reused)
```

```
    LcdDisplay/LcdDisplay.h, .cpp  # ECAL: I2C LCD driver (reused)
    StdioSerial/StdioSerial.h,.cpp # ECAL: STDIO redirection (reused)
    BinaryConditioner/             # SRV: debounce filter
    SignalConditioner/             # SRV: analog pipeline (reused)
  platformio.ini                   # env:lab4_1 configuration
  wokwi/lab4.1/
    diagram.json                   # Wokwi circuit definition
    wokwi.toml                     # Firmware path for simulation
```

2.9. Modular Implementation

Relay Driver Module

The `Relay` library provides a hardware abstraction for controlling a relay module via a digital GPIO pin. It supports both active-HIGH and active-LOW relay modules through a constructor parameter. The interface exposes four methods: `init()` configures the pin as OUTPUT and deactivates the relay, `turnOn()` and `turnOff()` set the relay state with automatic logic inversion for active-LOW modules, and `setState(bool)` provides a convenient boolean-controlled interface. The `isOn()` getter returns the current relay state for diagnostic reporting.

Listing 2.2 *– Relay driver interface (Relay.h)*

```
class Relay {
public:
    Relay(uint8_t pin, bool activeHigh = true);
    void init();
    void turnOn();
    void turnOff();
    void setState(bool on);
    bool isOn() const;
};
```

PWM Actuator Driver Module

The `PwmActuator` library abstracts PWM output control for analog actuators. It accepts a duty cycle as a percentage (0.0–100.0%) and converts it to the 0–255 range used by Arduino's `analogWrite()` function. Input values are automatically clamped to the valid range. The `getDutyCycle()` and `getRawPwm()` getters provide diagnostic access to the current output state.

Listing 2.3 *– PWM actuator driver interface (PwmActuator.h)*

```
class PwmActuator {
public:
    PwmActuator(uint8_t pin);
    void init();
    void setDutyCycle(float percent);
    float getDutyCycle() const;
```

```
    uint8_t getRawPwm() const;
};
```

Binary Conditioner Module

The `BinaryConditioner` library implements counter-based debouncing as described in Section 4.1.6 of the lab manual. A counter increments when the raw input is HIGH and decrements when LOW, bounded by [0, MAX]. The output transitions to ON only when the counter reaches MAX, and to OFF only when it reaches zero. This prevents false triggering from transient noise or accidental rapid command changes. The module exposes diagnostic getters for the counter value and transitioning status.

Listing 2.4 *– Binary conditioner interface (BinaryConditioner.h)*

```
class BinaryConditioner {
public:
    BinaryConditioner(uint8_t maxCount = 5);
    void init();
    bool process(bool rawInput);
    bool getState() const;
    uint8_t getCounter() const;
    bool isTransitioning() const;
};
```

Reused Modules

The following library modules from previous labs are reused without modification:

- **SignalConditioner** — three-stage pipeline (saturation → median → EWMA) for conditioning the analog actuator setpoint.
- **LcdDisplay** — I2C LCD driver with `showTwoLines()` for displaying both actuator states.
- **Led** — GPIO LED driver for status indicators.
- **StdioSerial** — STDIO redirection over UART for `printf()` output.

3. Obtained Results

3.1. Build Results

The project was compiled successfully using PlatformIO for the `lab4_1` environment targeting the Arduino Mega 2560. The build output confirmed successful compilation with the following resource utilization: RAM usage of 16.2% (1324 bytes out of 8192 bytes) and Flash usage of 8.5% (21472 bytes out of 253952 bytes). The relatively low resource usage demonstrates that

the FreeRTOS-based multi-task architecture with four concurrent tasks, mutex synchronization, and signal conditioning pipelines fits comfortably within the ATmega2560's memory constraints.

[Screenshot pending — save to resources/ObtainedResults/build_results.png]
Show PlatformIO terminal with successful build output for env:lab4_1

***Figure 3.1** – PlatformIO build output showing successful compilation for Lab 4.1*

3.2. Simulation Startup

Upon starting the Wokwi simulation, the system initializes all hardware components and displays the startup message on the LCD ("Lab 4.1: Dual / Actuator Ctrl") and on the serial terminal. The relay is initially OFF, the PWM duty cycle is 0%, and all status LEDs are off. The serial terminal prints the command help message listing available commands.

[Screenshot pending — save to resources/ObtainedResults/simulation_start.png]
Show Wokwi simulation with LCD showing startup text and serial terminal showing help

***Figure 3.2** – Wokwi simulation at startup with initial state displayed*

3.3. Test Results

TC1 — Binary ON Command

Sending the "ON" command via the serial terminal triggers the binary actuator control pipeline. The command task updates the target state, and the binary task begins incrementing the debounce counter. After 5 consecutive confirmations (500 ms), the relay activates, the green status LED turns on, and the LCD updates to show "Rly:ON". The serial report confirms the state transition and shows the debounce counter at 5/5.

[Screenshot pending — save to resources/ObtainedResults/test_binary_on.png]
Show "ON" command sent, relay activated, green LED on, LCD showing Rly:ON

***Figure 3.3** – Binary actuator ON command test — relay activated after debounce*

TC2 — Binary OFF Command

After the relay is active, sending "OFF" initiates the reverse debounce process. The counter decrements from 5 to 0 over 500 ms, after which the relay deactivates and the green LED turns off.

[Screenshot pending — save to resources/ObtainedResults/test_binary_off.png]
Show "OFF" command sent, relay deactivated, green LED off, LCD showing Rly:OFF

***Figure 3.4** – Binary actuator OFF command test — relay deactivated after debounce*

TC4 — Analog Speed Command

Sending "SPEED 75" sets the analog actuator target to 75%. The analog task applies ramping (5%/cycle, 100 ms period) so the duty cycle increases gradually over approximately 1.5 seconds. The signal conditioning pipeline further smooths the transition. The blue PWM LED brightens progressively, and the LCD shows the duty cycle approaching 75%.

[Screenshot pending — save to resources/ObtainedResults/test_analog_speed.png]
Show "SPEED 75" command, PWM ramping up, serial reports showing gradual increase

***Figure 3.5** – Analog actuator speed command test — PWM ramping to 75%*

TC10 — Dual Actuator Simultaneous Operation

Sending "ON" followed by "SPEED 50" demonstrates independent operation of both actuators. The relay activates through its debounce pipeline while the PWM simultaneously ramps to 50%. The LCD displays both states on a single screen, confirming that the FreeRTOS tasks operate independently without interference.

[Screenshot pending — save to resources/ObtainedResults/test_dual_actuator.png]
Show both actuators active: relay ON + PWM at 50%, LCD showing both states

***Figure 3.6** – Dual actuator simultaneous operation test*

3.4. Serial Report Output

The display task prints a structured actuator report every 500 ms, showing the binary state with debounce diagnostics and the analog state with duty cycle, raw setpoint, and PWM value. This output can be used with the Arduino Serial Plotter for real-time visualization of the analog actuator response.

[Screenshot pending — save to resources/ObtainedResults/test_report.png]
Show serial monitor with multiple actuator report cycles

***Figure 3.7** – Serial monitor output showing periodic actuator reports*

4. Conclusions

4.1. Performance Analysis

The dual actuator control system successfully demonstrates Varianta C of the laboratory assignment, achieving simultaneous control of a binary actuator (relay) and an analog actuator (PWM) with independent signal conditioning pipelines and unified LCD reporting.

The binary actuator's counter-based debouncing provides reliable switching with a 500 ms confirmation period (5 cycles at 100 ms). This is appropriate for relay control, where the mechanical switching time of a typical relay (5–15 ms) is well within the debounce window. The debounce filter effectively rejects rapid command toggling that could cause relay chatter and premature contact wear.

The analog actuator's conditioning pipeline combines ramping (5%/cycle rate limit) with the three-stage SignalConditioner (saturation, median, EWMA). The ramping stage limits the rate of duty cycle change to 50%/second, providing smooth transitions that protect mechanical actuators from sudden load changes. The SignalConditioner further smooths the ramped value, achieving a stable output even with noisy or rapidly changing command inputs.

The FreeRTOS multi-task architecture provides clean separation of concerns: the command task handles serial I/O at the highest priority (50 ms period), ensuring responsive command processing well within the 100 ms latency requirement. The actuator tasks run at medium priority (100 ms period) for timely signal conditioning, while the display task runs at the lowest priority (500 ms period) since visual updates are less time-critical. The mutex-protected shared state ensures data consistency between tasks without deadlock risk, as all critical sections are short and non-blocking.

4.2. Limitations

The current implementation has several limitations that could be addressed in future improvements. The analog actuator is simulated using a PWM-driven LED rather than a real DC motor, so characteristics such as back-EMF, inertia, and current limiting are not represented. The serial command interface is text-based and single-character buffered, which could lose characters under very high input rates. The LCD update rate of 500 ms is sufficient for human observation but may miss transient states during rapid testing. The system does not implement overload protection or fault detection for the actuators.

4.3. Improvement Ideas

Several enhancements could extend the system's capabilities. Adding a motor driver (L298N or L293D) would enable real DC motor control with direction switching and current sensing. Implementing a PID controller for the analog actuator would enable closed-loop speed or position control. Adding EEPROM storage for configuration parameters (debounce count, ramp rate, conditioning parameters) would allow persistent customization. Integrating a keypad as an alternative input device would provide standalone operation without a serial terminal connection.

4.4. Real-World Impact

Actuator control with signal conditioning is fundamental to virtually every automated system. The techniques implemented in this lab — debounced relay switching, ramped PWM control, and multi-task coordination — are directly applicable to industrial automation (PLC-based

production lines), building management systems (HVAC control, lighting automation), automotive systems (window motors, seat positioning, climate control), and consumer electronics (smart home devices, appliance control). The FreeRTOS-based architecture demonstrates the standard approach used in commercial embedded products, where multiple independent control loops must operate concurrently with guaranteed timing and data integrity.

5. Note Regarding the Usage of AI Tools

During the writing of this report, the author used Claude (Anthropic) to generate and consolidate content. The results were reviewed, validated, and adjusted according to the lab requirements.

6. Bibliography

1. A. Bragarenco, V. Astafi, "Sisteme Electronice Încorporate — Indicaţii metodice pentru lucrări de laborator," Universitatea Tehnică a Moldovei, Chişinău, 2025. ISBN 978-9975-45-642-5.

2. Arduino Reference — `analogWrite()`, `https://www.arduino.cc/reference/en/language/functions/analog-io/analogwrite/`. Accessed: March 2026.

3. FreeRTOS Documentation — Semaphores and Mutexes, `https://www.freertos.org/a00113.html`. Accessed: March 2026.

4. R. Barry, "Mastering the FreeRTOS Real Time Kernel — A Hands-On Tutorial Guide," Real Time Engineers Ltd., 2016.

5. Wokwi Documentation — Relay Module, `https://docs.wokwi.com/parts/wokwi-relay-module` Accessed: March 2026.

6. ATmega2560 Datasheet, Microchip Technology Inc., `https://ww1.microchip.com/downloads/en/devicedoc/atmel-2549-8-bit-avr-microcontroller-atmega640-1280-1281-2560-256datasheet.pdf`. Accessed: March 2026.

7. Appendix — Source Code

The complete source code for this lab is available in the GitHub repository:
GitHub Repository: `https://github.com/avremere/es-labs`

7.1. Configuration Header (lab4_1_config.h)

***Listing 7.1** – Pin mapping and configuration constants*

```
#ifndef LAB4_1_CONFIG_H
#define LAB4_1_CONFIG_H

// Pin Mapping
#define PIN_RELAY           7    // Binary actuator: relay module
#define PIN_PWM_ACTUATOR    9    // Analog actuator: PWM output
#define PIN_LED_BINARY     12    // Green LED: relay state indicator
#define PIN_LED_ANALOG     11    // Yellow LED: analog activity indicator

// LCD I2C
#define LCD_I2C_ADDR     0x27
#define LCD_COLS           16
#define LCD_ROWS            2

// Serial
#define SERIAL_BAUD      9600

// Binary conditioner: debounce confirmations
#define BINARY_DEBOUNCE_COUNT   5

// Analog conditioner parameters
#define ANALOG_MEDIAN_WINDOW    5
#define ANALOG_EWMA_ALPHA     0.3f
#define ANALOG_MIN_CLAMP      0.0f
#define ANALOG_MAX_CLAMP    100.0f
#define ANALOG_RAMP_RATE      5.0f  // max change per cycle (%)

// Task timing (ms)
#define TASK_COMMAND_PERIOD_MS     50
#define TASK_BINARY_PERIOD_MS     100
#define TASK_ANALOG_PERIOD_MS     100
#define TASK_DISPLAY_PERIOD_MS    500

// FreeRTOS stack sizes (words)
#define STACK_COMMAND    192
#define STACK_BINARY     128
#define STACK_ANALOG     128
#define STACK_DISPLAY    256

#endif // LAB4_1_CONFIG_H
```

7.2. Lab Entry Point (lab4_1_main.cpp)

***Listing 7.2** – FreeRTOS task orchestration for dual actuator control*

```
#include <Arduino.h>
```

```
#include <Arduino_FreeRTOS.h>
#include <semphr.h>
#include "lab4_1_config.h"
#include "Relay.h"
#include "PwmActuator.h"
#include "BinaryConditioner.h"
#include "SignalConditioner.h"
#include "LcdDisplay.h"
#include "Led.h"
#include "StdioSerial.h"

// Hardware instances
static Relay         relay(PIN_RELAY, true);
static PwmActuator   pwmAct(PIN_PWM_ACTUATOR);
static Led           ledBinary(PIN_LED_BINARY);
static Led           ledAnalog(PIN_LED_ANALOG);
static LcdDisplay    lcd(LCD_I2C_ADDR, LCD_COLS, LCD_ROWS);

// Signal conditioners
static BinaryConditioner binCond(BINARY_DEBOUNCE_COUNT);
static SignalConditioner anlCond(ANALOG_MEDIAN_WINDOW,
    ANALOG_EWMA_ALPHA, ANALOG_MIN_CLAMP, ANALOG_MAX_CLAMP);

// Shared state (mutex-protected)
static SemaphoreHandle_t stateMutex;
static volatile bool  cmdBinaryTarget = false;
static volatile float cmdAnalogTarget = 0.0f;
static volatile bool  actBinaryState  = false;
static volatile float actAnalogDuty   = 0.0f;

// Command parsing updates shared targets under mutex.
// Binary task applies debounce, drives relay + LED.
// Analog task applies ramping + conditioning, drives PWM.
// Display task reads all state, updates LCD + serial.

void lab4_1Setup() {
    stdioSerialInit(SERIAL_BAUD);
    relay.init(); pwmAct.init();
    ledBinary.init(); ledAnalog.init();
    lcd.init();
    binCond.init(); anlCond.reset();
    stateMutex = xSemaphoreCreateMutex();
    // Create 4 FreeRTOS tasks with priorities 3,2,2,1
    xTaskCreate(commandTask,"Cmd", STACK_COMMAND,NULL,3,NULL);
    xTaskCreate(binaryTask, "Bin", STACK_BINARY, NULL,2,NULL);
    xTaskCreate(analogTask, "Anl", STACK_ANALOG, NULL,2,NULL);
    xTaskCreate(displayTask,"Dsp", STACK_DISPLAY,NULL,1,NULL);
}
```

```
void lab4_1Loop() { /* FreeRTOS scheduler runs */ }
```

7.3. Relay Driver (Relay.h / Relay.cpp)

Listing 7.3 *– Relay driver implementation*

```
// Relay.h
class Relay {
public:
    Relay(uint8_t pin, bool activeHigh = true);
    void init();        // Configure pin, set OFF
    void turnOn();      // Activate relay
    void turnOff();     // Deactivate relay
    void setState(bool on);
    bool isOn() const;
private:
    uint8_t _pin; bool _activeHigh; bool _state;
};

// Relay.cpp
void Relay::init() {
    pinMode(_pin, OUTPUT); turnOff();
}
void Relay::turnOn() {
    digitalWrite(_pin, _activeHigh ? HIGH : LOW);
    _state = true;
}
void Relay::turnOff() {
    digitalWrite(_pin, _activeHigh ? LOW : HIGH);
    _state = false;
}
```

7.4. PWM Actuator Driver (PwmActuator.h / PwmActuator.cpp)

Listing 7.4 *– PWM actuator driver implementation*

```
// PwmActuator.h
class PwmActuator {
public:
    PwmActuator(uint8_t pin);
    void init();
    void setDutyCycle(float percent); // 0-100%
    float getDutyCycle() const;
    uint8_t getRawPwm() const;       // 0-255
private:
    uint8_t _pin; float _dutyCycle;
```

```
};

// PwmActuator.cpp
void PwmActuator::setDutyCycle(float percent) {
    if (percent < 0.0f) percent = 0.0f;
    if (percent > 100.0f) percent = 100.0f;
    _dutyCycle = percent;
    uint8_t pwmVal = (uint8_t)(percent * 255.0f / 100.0f);
    analogWrite(_pin, pwmVal);
}
```

7.5. Binary Conditioner (BinaryConditioner.h / BinaryConditioner.cpp)

Listing 7.5 *– Counter-based debounce filter implementation*

```
// BinaryConditioner.h
class BinaryConditioner {
public:
    BinaryConditioner(uint8_t maxCount = 5);
    void init();
    bool process(bool rawInput);  // Returns stable output
    bool getState() const;
    uint8_t getCounter() const;
    bool isTransitioning() const;
private:
    uint8_t _maxCount, _counter; bool _output;
};

// BinaryConditioner.cpp
bool BinaryConditioner::process(bool rawInput) {
    if (rawInput) {
        if (_counter < _maxCount) _counter++;
    } else {
        if (_counter > 0) _counter--;
    }
    if (_counter >= _maxCount) _output = true;
    else if (_counter == 0)    _output = false;
    return _output;
}
```